Assignment-2

학과 : 로봇공학과
학번 : 2025077101
이름 : 임현진

1. And 및 Or 논리 회로

4개의 입력을 이용하여 and 연산과 or 연산을 수행하는 회로를 설계하였다. and 연산은 모든 입력이 1일 때 1을 출력하고 or 연산은 하나 이상의 입력이 1일 때 1을 출력한다.

-and

a	b	output
0	0	0
0	1	0
1	0	0
1	1	1

-or

a	b	output
0	0	0
0	1	1
1	0	1
1	1	1

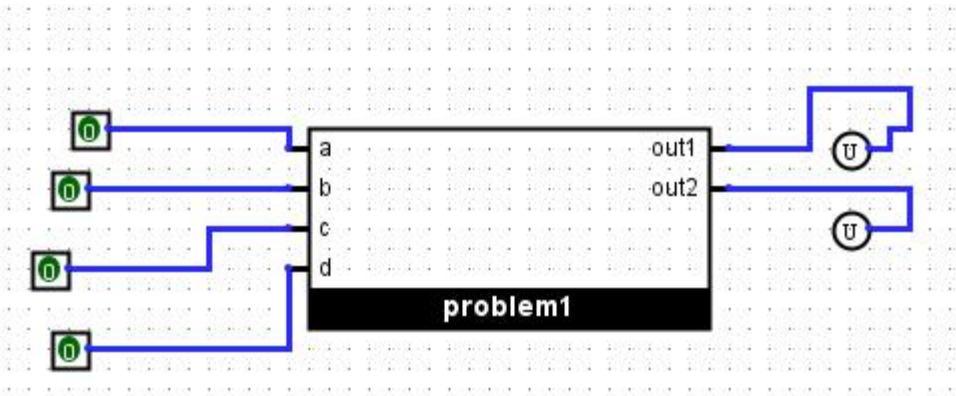
-VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

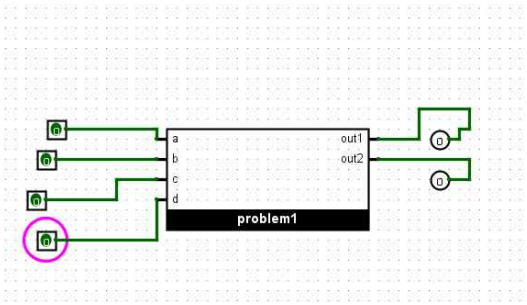
entity problem1 is
Port (
a    : in  STD_LOGIC;
b    : in  STD_LOGIC;
c    : in  STD_LOGIC;
d    : in  STD_LOGIC;
out1 : out STD_LOGIC;
out2 : out STD_LOGIC
);
end problem1;

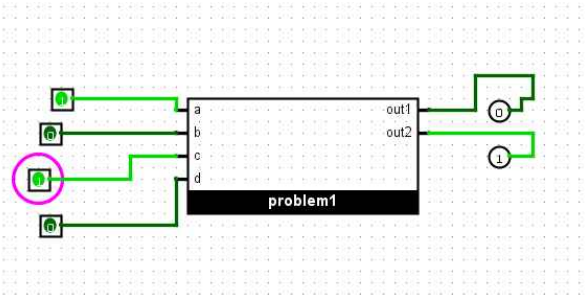
architecture Behavioral of problem1 is
begin
out1 <= a and b and c and d;
out2 <= a or b or c or d;
end Behavioral;
```

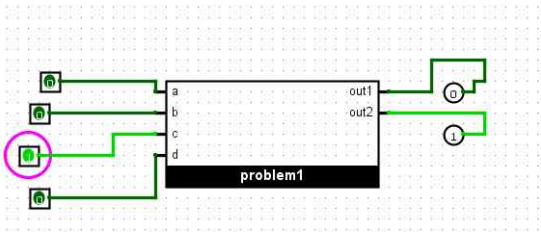
-Import and simulate it in Logisim-evolution

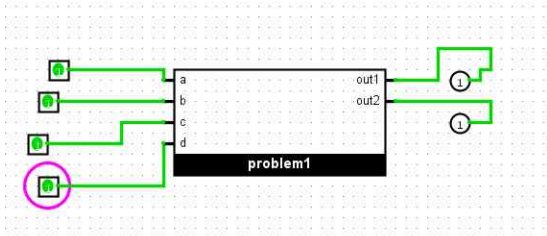


-Verify









out1은 and 연산 결과, out2는 or 연산 결과이다. 모든 입력이 0일 때 AND와 OR 출력은 모두 0으로 나타났다. 모든 입력이 1일 때는 두 출력이 모두 1로 나타났다. 이를 통해 AND 및 OR 논리 연산이 정상적으로 수행됨을 확인하였다.

2. 8-to-1 멀티플렉서

8개의 입력 중 선택 신호에 해당하는 하나의 입력을 출력하는 8-to-1 멀티플렉서를 설계하였다. 3비트 선택 신호를 이용하여 총 8개의 입력 중 하나를 선택할 수 있다.

-VHDL code

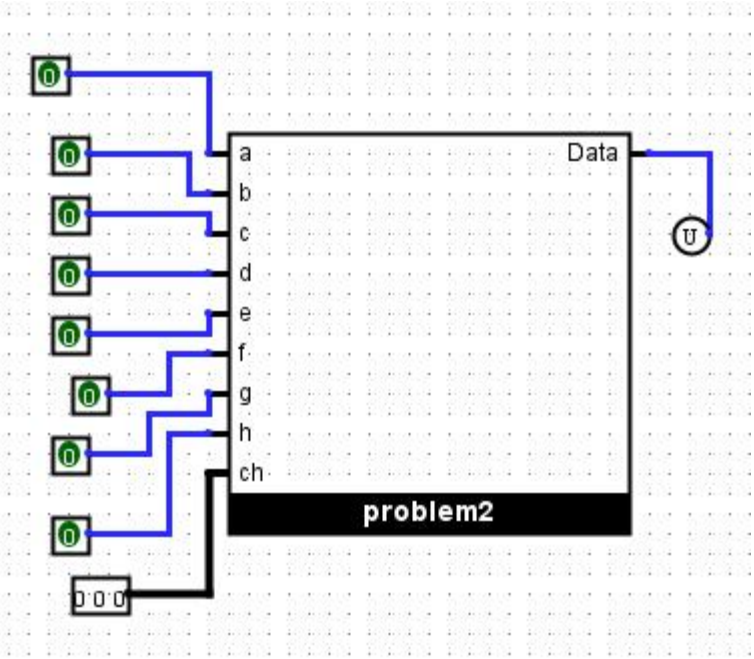
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity problem2 is
Port (
a  : in  STD_LOGIC;
b  : in  STD_LOGIC;
c  : in  STD_LOGIC;
d  : in  STD_LOGIC;
e  : in  STD_LOGIC;
f  : in  STD_LOGIC;
g  : in  STD_LOGIC;
h  : in  STD_LOGIC;
ch : in  STD_LOGIC_VECTOR(2 downto 0);
Data : out STD_LOGIC
);
end problem2;

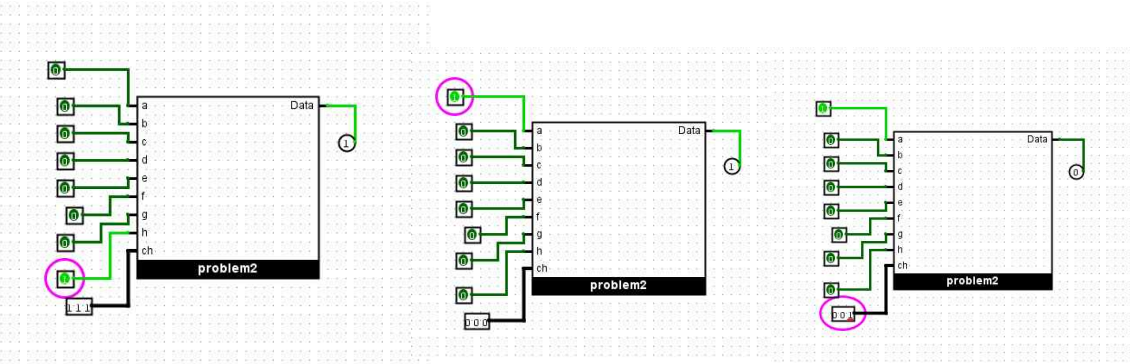
architecture Behavioral of problem2 is
begin
process(a, b, c, d, e, f, g, h, ch)
begin
case ch is
when "000" => Data <= a;
when "001" => Data <= b;
```

```
when "010" => Data <= c;
when "011" => Data <= d;
when "100" => Data <= e;
when "101" => Data <= f;
when "110" => Data <= g;
when "111" => Data <= h;
when others => Data <= '0';
end case;
end process;
end Behavioral;
```

-Import and simulate it in Logisim-evolution



-Verify



선택 신호에 따라 지정된 입력값이 출력으로 전달되는 것을 확인하였다. 이를 통해 8개의 입력 중 하나를 선택하는 멀티플렉서의 기능이 정상적으로 수행됨을 확인하였다.

3. 7세그먼트 디스플레이를 이용한 문자 표시

7개의 세그먼트를 조합하여 문자 a, b, c, d, f, g를 표시하는 회로를 설계하였다. 선택 신호

에 따라 각 세그먼트의 출력 패턴을 변경하여 원하는 문자를 표현한다.

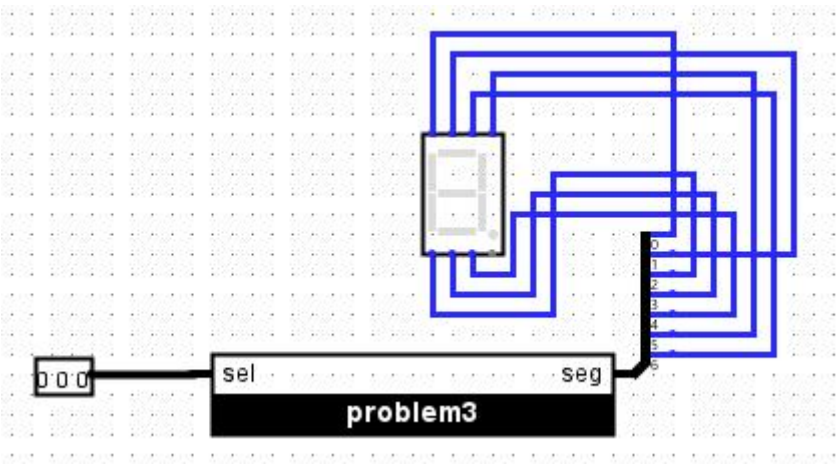
-VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

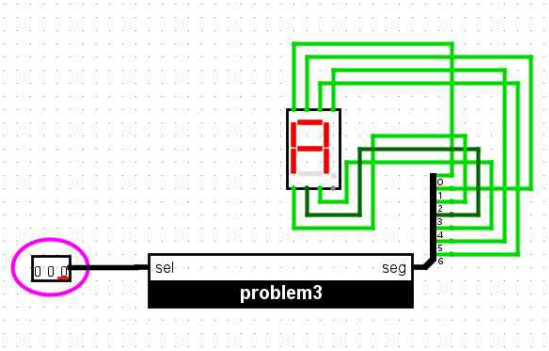
entity problem3 is
Port (
sel : in  STD_LOGIC_VECTOR(2 downto 0);
seg : out STD_LOGIC_VECTOR(6 downto 0)
);
end problem3;

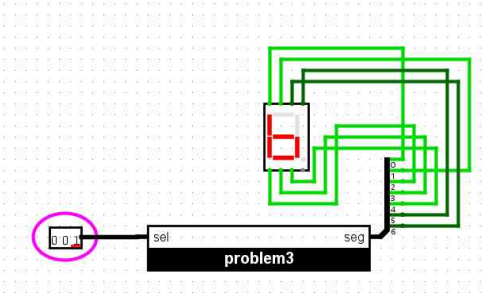
architecture Behavioral of problem3 is
begin
process(sel)
begin
case sel is
when "000" => seg <= "1110111"; -- a
          when "001" => seg <= "0011111"; -- b
          when "010" => seg <= "0001101"; -- c
          when "011" => seg <= "0111101"; -- d
          when "100" => seg <= "1000111"; -- f
          when "101" => seg <= "1111011"; -- g
          when others => seg <= "0000000";
end case;
end process;
end Behavioral;
```

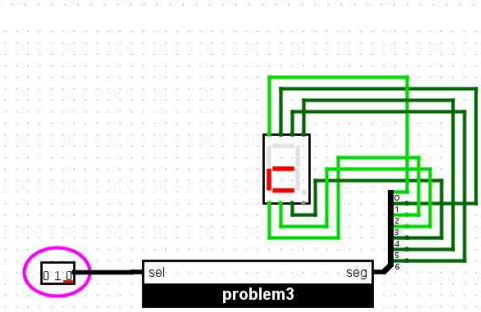
-Import and simulate it in Logisim-evolution

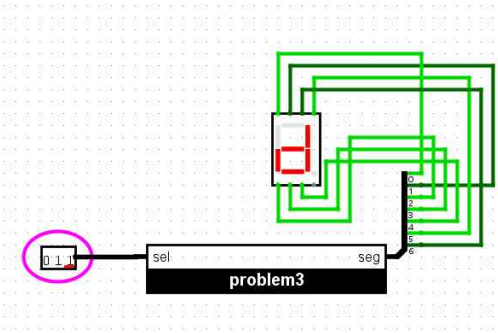


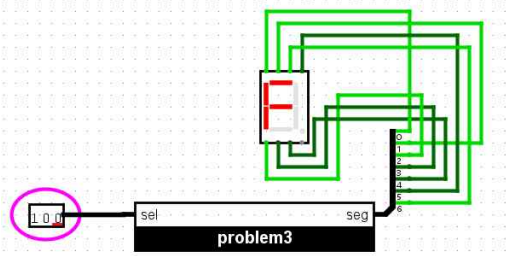
-Verify

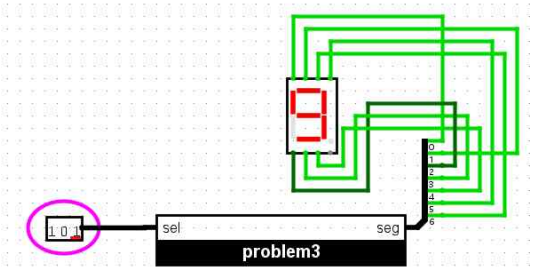












선택 신호를 변경함에 따라 7세그먼트 출력 패턴이 변경되는 것을 확인하였다. 이를 통해 각 문자에 해당하는 세그먼트 조합을 출력하는 기능이 구현되었음을 확인하였다.

4. 25-persion voting machine

25명의 투표 결과를 처리하는 회로를 설계하였다. 찬성표가 전체 투표자의 과반수인 13표 이상이면 출력이 1이 되고, 12표 이하이면 출력이 0이 된다.

-VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity problem4 is
Port (
vt     : in  STD_LOGIC_VECTOR(24 downto 0);
result : out STD_LOGIC
);
end problem4;

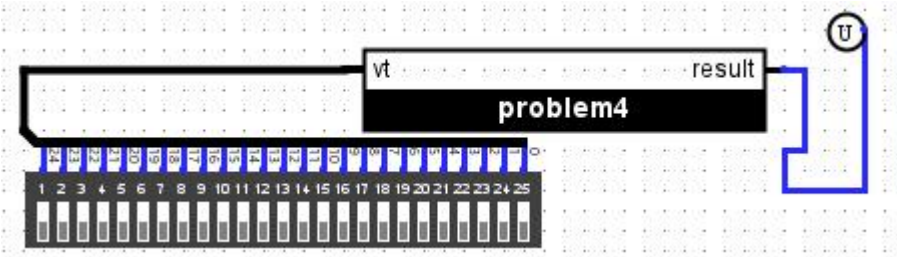
architecture Behavioral of problem4 is
```

```
begin
process(vt)
variable count_ones : integer range 0 to 25;
begin
count_ones := 0;

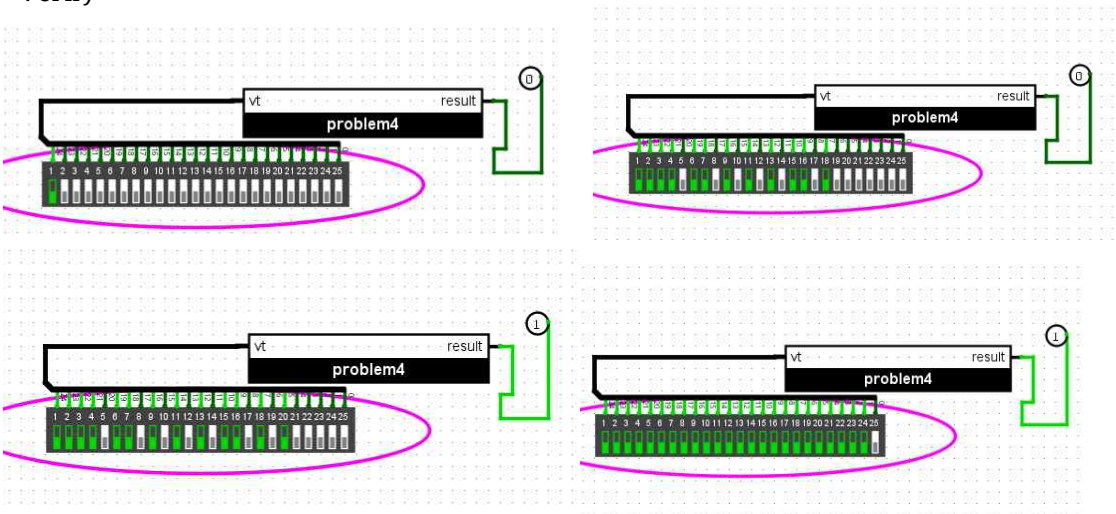
for i in 0 to 24 loop
if vt(i) = '1' then
count_ones := count_ones + 1;
end if;
end loop;

if count_ones >= 13 then
result <= '1';
else
result <= '0';
end if;
end process;
end Behavioral;
```

-Import and simulate it in Logisim-evolution



-Verify



찬성표가 1개일 때와 12개일 때는 출력이 0으로 나타났으며, 13개일 때와 24개일 때는 출력이 1로 나타났다. 이를 통해 과반수 기준에 따라 투표 결과가 정상적으로 결정됨을 확인하였다.

5. 13비트 입력의 1 개수 계산 및 7세그먼트 표시

13비트 입력에서 값이 1인 비트의 개수를 계산하고, 그 결과를 7세그먼트 디스플레이에 표시하는 회로를 설계하였다. 활성화 신호 en이 1이면 계산 결과를 표시하고, 0이면 디스플레이를 끈다.

-VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

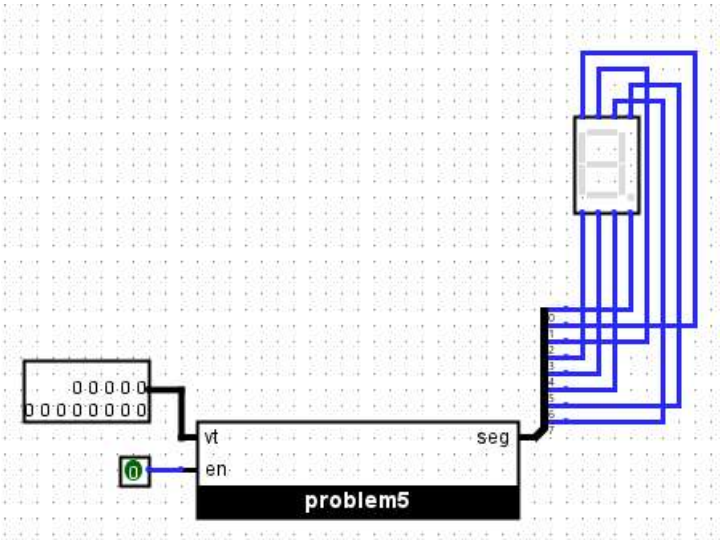
entity problem5 is
Port (
vt  : in  STD_LOGIC_VECTOR(12 downto 0);
en  : in  STD_LOGIC;
seg : out STD_LOGIC_VECTOR(7 downto 0)
);
end problem5;

architecture Behavioral of problem5 is
begin
process(vt, en)
variable count_ones : integer range 0 to 13;
begin
count_ones := 0;

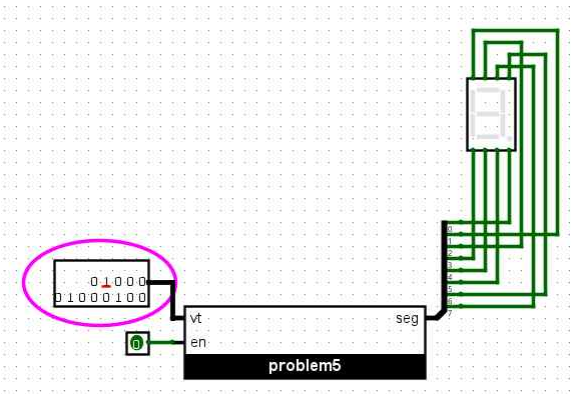
for i in 0 to 12 loop
if vt(i) = '1' then
count_ones := count_ones + 1;
end if;
end loop;

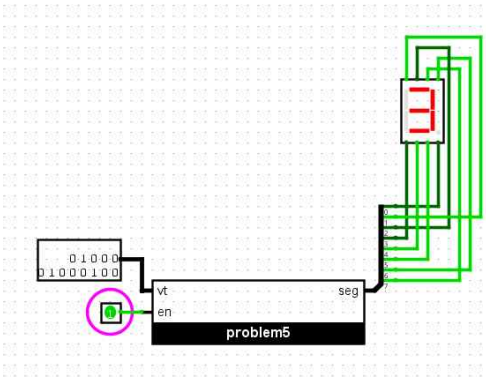
if en = '0' then
seg <= "00000000";
else
case count_ones is
when 0  => seg <= "11111100";
when 1  => seg <= "01100000";
when 2  => seg <= "11011010";
when 3  => seg <= "11110010";
when 4  => seg <= "01100110";
when 5  => seg <= "10110110";
when 6  => seg <= "10111110";
when 7  => seg <= "11100000";
when 8  => seg <= "11111110";
when 9  => seg <= "11110110";
when 10 => seg <= "11101110"; -- A
              when 11 => seg <= "00111110"; -- b
              when 12 => seg <= "10011100"; -- C
              when 13 => seg <= "01111010"; -- d
              when others => seg <= "00000000";
end case;
end if;
end process;
end Behavioral;
```

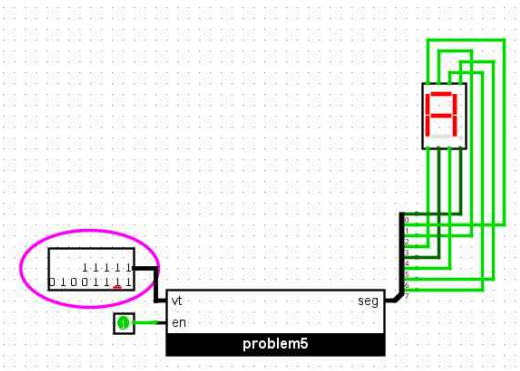
-Import and simulate it in Logisim-evolution



-Verify







입력 벡터에서 1의 개수를 계산하고, 해당 개수에 맞는 7세그먼트 출력 패턴을 생성하도록 구현하였다. 또한 en이 0일 때 디스플레이가 꺼지고, 1일 때 계산 결과가 표시되는 것을 확인하였다. 13비트 입력에서는 1의 개수가 최대 13개이므로 실제로 표현 가능한 결과는 0~9와 A~D이다.